
GRIDVERITAS

ARCHITECTURE BASELINE

Version 0.2 | July 2026

Prepared by Oliver Bodemer

TABLE OF CONTENTS

1. Purpose of This Document.....	3
2. Domain Brief (US Energy Focus).....	3
2.1 Operational Context.....	3
2.2 Regulatory Drivers (US).....	3
2.3 Core Problem (Domain View).....	4
2.4 Target Users & Value.....	4
2.5 Domain Assumptions (to validate).....	4
3. Architecture Baseline.....	5
3.1 Architectural Principles.....	5
3.2 High-Level Components.....	5
3.3 Data Model Outline (PostgreSQL).....	6
3.4 Initial REST API Surface (Core).....	7
3.5 Security Outline.....	8
4. Architecture Decision Records (Initial).....	9
ADR-001 — Core platform: Spring Boot + PostgreSQL.....	9
ADR-002 — Edge agents: Go.....	9
ADR-003 — UI: React.....	9
ADR-004 — AI: Ollama + LangChain for audit assistance (not anomaly detection).....	9
ADR-005 — Deployment: Docker + Jenkins on Debian.....	9
ADR-006 — Provenance & anchoring: Merkle + RFC 3161, no blockchain.....	10
5. Implementation Sequence (from this baseline).....	11
6. Open Points for Domain Discovery.....	12
7. Immediate Next Steps.....	12
8. Revision History.....	13

1. PURPOSE OF THIS DOCUMENT

This document consolidates the Domain Brief (M1) and the Architecture Baseline (M2). It is the foundation for implementation. It records the current understanding of the US energy-domain context, the target problem, and the technical architecture that will be built: the Spring Boot core, PostgreSQL storage, Merkle provenance with external anchoring, Go edge agents, the React UI, and the AI layer.

Status: Working baseline, aligned with the Project Initiation Document (v0.9.1). Domain findings will be refined through discovery interviews and system observation. Architecture decisions are recorded as ADRs and can evolve, but changes after significant coding must be explicit and captured in a new or superseding ADR.

2. DOMAIN BRIEF (US ENERGY FOCUS)

2.1 OPERATIONAL CONTEXT

US bulk-electric-system and distribution operators rely on SCADA, EMS, historians, and increasingly on DER and edge telemetry. Data is used for real-time control decisions, market settlement, reliability reporting, and post-incident investigation. The same data is subject to NERC CIP cybersecurity requirements and growing expectations for demonstrable integrity.

Data paths are typically multi-vendor and often span electronic security perimeters, passing through gateways, historians, and multiple intermediary systems between the source device and the system that finally consumes the reading. This is the operational setting in which the integrity problem (Section 2.3) arises.

2.2 REGULATORY DRIVERS (US)

- NERC CIP: NERC develops and enforces the Critical Infrastructure Protection standards; electronic security perimeter, system security management, and incident-evidence requirements create demand for stronger provenance and auditability of operational data.
- FERC: Approves and directs the reliability standards and issues cybersecurity orders that shape utility obligations around data and systems affecting the bulk power system.
- DOE / CESER guidance: Sector-specific cybersecurity practices, coordination, and information sharing.
- NIST Cybersecurity Framework (CSF 2.0) & SP 800 series: Widely adopted risk-management and control language used by energy entities (notably SP 800-82 Rev. 3 for OT security).
- State PUCs: Increasing attention to cybersecurity posture and integrity evidence for regulated utilities.

2.3 CORE PROBLEM (DOMAIN VIEW)

Operators cannot continuously and efficiently prove that operational data has not been tampered with or silently degraded between source and decision point. Consequences include elevated operational and safety risk when automated systems act on unverified data, high cost and latency of audits and forensics, difficulty integrating multi-vendor and multi-party data under a common trust model, and limited ability to demonstrate integrity to regulators, auditors, insurers, and partners.

2.4 TARGET USERS & VALUE

- **Utility / ISO OT & cybersecurity teams:** Real-time or near-real-time verification of critical telemetry; faster incident investigation; stronger audit artifacts.
- **Compliance / internal audit:** Evidence that can support CIP-related controls and incident reporting.
- **Future external auditors / insurers:** Independent verification of data-provenance claims — including externally anchored proofs that hold even without trusting the platform operator.

2.5 DOMAIN ASSUMPTIONS (TO VALIDATE)

These are working assumptions to be confirmed or narrowed during M1 domain discovery. Where an assumption also represents something that must be actively provided, it is tracked as a dependency in the Project Document (PID 9.3).

- Representative telemetry or simulation data can be obtained for development and pilot.
- Edge or gateway devices can run a lightweight Go agent, or can forward data to one.
- Existing identity/PKI, or a project-specific PKI, can be used for agent and service identity.
- Public RFC 3161 timestamping authorities remain available and acceptable as an external anchor for the MVP phase.
- Formal NERC CIP certification is out of scope for the first release; producing useful evidence and controls is in scope.

3. ARCHITECTURE BASELINE

3.1 ARCHITECTURAL PRINCIPLES

- Sit alongside existing SCADA/EMS/historians — do not replace them.
- Cryptographic attestation at or near the source; verification available to any authorized consumer.

- Provenance is tamper-evident by construction (hash-linked / Merkle) and externally anchored, so recorded history cannot be altered undetected — including by the platform operator.
- Edge resilience: agents continue local attestation under network partition.
- Open API first; adapters for common OT protocols.
- Security and auditability designed with NERC CIP and NIST CSF language in mind.
- Incremental delivery: first vertical slice = Spring Boot core + basic REST + React UI.
- No blockchain: trust is established by source signatures and external anchoring, not by a distributed-ledger or consensus protocol.

3.2 HIGH-LEVEL COMPONENTS

- Go Edge Agent: Runs on gateway / edge node. Signs data points or batches (payload + timestamp + source identity + sequence). Submits attestations to the core. Buffers under partition.
- Spring Boot Core (Verification Fabric): Receives attestations, maintains hash-linked / Merkle provenance, periodically computes Merkle roots and submits them for external anchoring, stores proofs and metadata in PostgreSQL, exposes REST verification and query APIs, hosts the policy engine.
- PostgreSQL: Primary store for sources, attestations, Merkle roots, anchors, verification events, policies, and the audit log (see 3.3).
- External Timestamping Authority (TSA): One or more independent RFC 3161 services that sign submitted Merkle roots, providing the external anchor that makes recorded history tamper-evident against the operator. Not operated by the project.
- React Console: Operator-facing UI for viewing attestations, running verification checks, inspecting audit trails, and basic policy/config.
- AI Services:
 - Statistical anomaly detection on attestation metadata, off the critical verification path (implemented): per-source sequence-gap, invalid-signature-rate, and source-silence signals, recorded as deduplicated findings in anomaly_findings.
 - Natural-language audit assistance via a local LLM (implemented, RAG-lite): facts are retrieved from the database and used to ground a local Ollama model; the model phrases answers over those facts only and is never used for per-event anomaly scoring. A semantic vector store is a later enhancement.
- Jenkins + Docker on Debian: Build, test, package, and deploy all services.

- API authentication & authorization (implemented): JWT bearer tokens (roles INGEST/ADMIN) for write operations, a static read key for read access; source registration and ingest are role-restricted. Anchor tokens are cryptographically verified (signature, message imprint, timestamping ECU, and — when a TSA CA is configured — trust-pinned to that CA).

3.3 DATA MODEL OUTLINE (POSTGRESQL)

Initial tables (subject to refinement). Attestation and provenance tables are append-only: the application role holds INSERT/SELECT only, with UPDATE/DELETE revoked. Attestation and provenance tables (attestations, merkle_roots, merkle_leaves) are append-only. Enforcement uses two database roles: schema migrations run as the table owner (full DDL), while the application connects as a non-owner role that is granted SELECT/INSERT only, with no UPDATE/DELETE on these tables. A non-owner role is required because a table owner always bypasses table privileges. sources remains mutable (its last_seen_at is updated at runtime). This stops accidental and normal-path mutation; deliberate operator tampering is caught by the external anchor (3.5), not by database permissions alone — append-only is defense-in-depth.

- sources / agents: source_id, public_key, status, last_seen. Identity and key material for each attesting agent.
- attestations: id, source_id, sequence (per-source, monotonic), payload_hash, src_timestamp (time claimed by the source), received_at (time recorded by the core), signature (Ed25519 over payload_hash + metadata), leaf_hash (hash of the canonical record; the Merkle leaf), raw_or_reference. Unique on (source_id, sequence).
- merkle_roots: id, root_hash, prev_root_hash (chains roots so whole periods cannot be dropped), leaf_count, computed_at. A root does not store an id range, because attestation IDs are UUIDs and therefore unordered.
- merkle_leaves: id, root_id (→ merkle_roots), leaf_index, att_id (→ attestations, unique — one root per attestation), leaf_hash. Written once when a root is sealed; this fixes membership and leaf ordering immutably and keeps attestations append-only (no back-reference is written onto the attestation).
- anchors: id, root_id (→ merkle_roots), authority (which TSA), token (signed RFC 3161 timestamp), anchored_at. The external evidence that a given root existed at a given time.
- anomaly_findings: id, source_id (nullable — global findings allowed), type (SEQUENCE_GAP | SIGNATURE_INVALID_SPIKE | SOURCE_SILENCE), severity, description, metric_value, window_start, window_end, detected_at, dedup_key (unique). Append-only; the dedup key prevents the same finding being recorded on every detection run.
- verification_events: id, event_type (VERIFY | PROOF), subject, result, principal, detail, created_at. Append-only. Records who verified what and the outcome.

- `audit_log`: id, action (SOURCE_CREATED | TOKEN_ISSUED | TOKEN_DENIED | ANCHOR_CREATED | ...), principal, target, detail, created_at. Append-only. Records security/configuration events.
- `verification_events`: who verified what, when, and the result. Feeds the audit trail.
- `policies`: rules for required attestation freshness, source allow-lists, and alerting thresholds.
- `audit_log`: tamper-evident record of configuration and policy changes.

The authoritative schema lives in versioned Flyway migrations (db/migration/, baseline V1); Hibernate runs in validate mode and no longer manages DDL. The append-only application role is provisioned by a separate one-time DBA script (db/roles/append_only_role.sql), since roles and privileges are cluster-level rather than part of a versioned migration.

The two-role split is now enforced at runtime: the application connects as the non-owner role `gridveritas_app`, which holds SELECT/INSERT on the append-only tables and additionally UPDATE on sources only; migrations run as the owner. The role and its grants are provisioned by a versioned migration (V5__app_role.sql), independent of database-init timing.

3.4 INITIAL REST API SURFACE (CORE)

All endpoints require authentication. Reads (and `/verify`, `/proof`, `/anomalies`, `/audit/ask`) accept either a JWT or an X-Read-Key header; writes require a JWT with the appropriate role.

Unauthenticated requests receive a JSON 401, insufficient-role requests a JSON 403.

- `POST /api/v1/auth/token` — exchange credentials for a JWT (`{token, role, expiresInSeconds}`); public.
- `POST /api/v1/attestations` — requires role INGEST or ADMIN.
- `POST /api/v1/sources` — requires role ADMIN (governed registration).

Minimum viable endpoints for the first Spring Boot slice:

- `POST /api/v1/attestations` — submit one or a batch of attestations.
- `GET /api/v1/attestations/{id}` — retrieve an attestation plus its verification status.
- `GET /api/v1/attestations/{id}/proof` — retrieve the verifiable proof for an attestation: its leaf hash, the Merkle path to the covering root, the root hash, and the external anchor (RFC 3161 token). This is what an independent party uses to verify without trusting the core.
- `POST /api/v1/verify` — request verification of a payload/hash against the stored chain.
- `GET /api/v1/roots/{id}` — retrieve a Merkle root and its anchor token.
- `GET /api/v1/sources` — list known agents/sources.
- `GET /api/v1/audit` — query recent verification and configuration events.
- `GET /api/v1/anomalies` — recent anomaly findings (optionally `?sourceId=`).
- `POST /api/v1/audit/ask` — natural-language audit question; returns the grounded answer plus the retrieved facts it was based on (`contextUsed`), or a graceful note if the assistant is unavailable.
- `GET /api/v1/audit` — recent security/configuration events (`audit_log`).
- `GET /api/v1/audit/verifications` — recent verification events.

All requests pass an early request guard: a per-client (IP) token-bucket rate limiter (a stricter limit on `/auth/token`) returning 429 with `Retry-After`, and a request-body size cap returning 413.

Behind a reverse proxy, the client IP is taken from `X-Forwarded-For`.

The proof response additionally reports `currentLeaf` (recomputed from the stored record's current fields) and `provenanceIntact` — whether that recomputed leaf still matches the externally anchored leaf. A mismatch demonstrates post-hoc tampering, caught against the anchored chain.

3.5 SECURITY OUTLINE

- Source authenticity: Digital signatures (Ed25519 per RFC 8032, or ECDSA) over a canonical message that binds the payload hash together with its metadata (source identity, sequence, source-claimed timestamp). Because the signature is created at the source and covers the metadata, authenticity is independent of the transport path, and stored metadata cannot be altered after the fact without invalidating the signature.

Canonical signed message. To guarantee that every client (Go edge agent, future SDKs) and the core produce identical bytes, the signed message is a length-prefixed binary structure — not signed JSON. It is constructed as followss:

- A fixed domain-separation tag and version: the ASCII string GridVeritas-Attestation-v1.
- Followed, in this exact order, by: `source_id` (canonical lowercase UUID, ASCII), `sequence` (int64, big-endian), `timestamp` (epoch milliseconds, int64, big-endian), and `payload_hash` (raw 32-byte SHA-256 of the payload).
- Each of the five fields — including the tag — is written as a 4-byte big-endian length prefix followed by the field bytes. There are no separators, so no field-injection ambiguity.

The Ed25519 signature is computed over these bytes. The timestamp is signed as an integer (epoch millis), never as a formatted string, to avoid cross-language format differences. Any client that cannot reproduce this construction byte-for-byte will fail verification. The attestation's leaf hash for the Merkle provenance (3.3) is SHA-256(0x00 || canonical message). Internal Merkle nodes are hashed as SHA-256(0x01 || left || right). These 0x00 / 0x01 domain-separation prefixes, together with an RFC 6962-style tree (splitting on the largest power of two below the leaf count), prevent second-preimage and node-as-leaf confusion.

- Provenance: Attestations are hash-linked into a Merkle structure; the core periodically computes a root over new leaves and links it to the previous root, so whole periods cannot be silently dropped. Roots are chained via `prev_root_hash`, and leaf membership and ordering are fixed at sealing time (see `merkle_leaves` in 3.3).
- External anchoring: Each Merkle root is submitted to one or more independent RFC 3161 timestamping authorities. The signed token attests the root existed at a given time; any later alteration changes the root and breaks the anchor, making tampering detectable — including by the platform operator. Within the single-node PostgreSQL baseline, provenance is tamper-evident, and the anchor is what extends that guarantee to cover the operator. Deliberately not a blockchain (see ADR-006).
- Verification (three checks): (1) the source signature over payload + metadata is valid; (2) the attestation's leaf reconstructs the stored Merkle root; (3) that root carries a valid external

anchor. Note: the anchor attests "existed no later than T," which — for attestations reconciled after a partition — may be later than the source timestamp.

- Transport: mTLS or equivalent between agents and core where the network allows.
- Authentication & authorization: the API is protected by Spring Security. JWTs (HS256, role claim) authorize write/admin operations; a constant-time-compared read key authorizes read access. Source registration is admin-only, closing the open-registration spoofing gap. Credentials for the MVP are configured users (admin, ingest); a directory/OIDC provider is a later step.
- Anchor verification: stored RFC 3161 tokens are verified — message imprint against the root, CMS signature against the embedded signer certificate, and the id-kp-timeStamping EKU. When a TSA CA certificate is configured, the signer is trust-pinned via a PKIX certificate-path build; otherwise integrity is verified but not trust-pinned (logged at startup). Verification runs both at acquisition (an unverifiable token is never stored) and on /proof retrieval.
- Key management: Hierarchical, with rotation support; hardware-backed keys (TPM / secure element) preferred when available.
- Threat model: A full threat model and cryptographic design document will be produced as a deliverable before pilot, referencing NIST CSF and NERC CIP concepts.
- Verification & audit trail: verification actions and security/configuration events are recorded append-only (verification_events, audit_log) with the authenticated principal, addressing repudiation and feeding the audit assistant. Recording is best-effort and runs in its own transaction so it never blocks the primary operation.
- Abuse resistance: a per-IP token-bucket rate limiter (general and a tighter auth limit) and a request-body size cap, enforced before the security chain so /auth/token is also protected. In-memory (single-host); a shared store is needed for multi-instance.
- Transport: mTLS is available for the agent→core ingest path (opt-in). The core serves HTTPS with client-auth: want; ingest additionally requires a validated client certificate, enforced at the application layer, while reads/UI use TLS with the read key / JWT. Browser TLS is typically terminated at the reverse proxy.

4. ARCHITECTURE DECISION RECORDS (INITIAL)

These ADRs lock the major technology and structural choices for the current phase. They can be superseded, but only with an explicit new ADR.

ADR-001 — CORE PLATFORM: SPRING BOOT + POSTGRESQL

Decision: Use Spring Boot for the verification engine and REST APIs; PostgreSQL as primary store.

Rationale: Mature ecosystem, strong fit for the solo developer's skills, good observability (Actuator), straightforward Docker packaging, reliable transactional storage for attestations and audit.

ADR-002 — EDGE AGENTS: GO

Decision: Implement attestation agents in Go.

Rationale: Small footprint, strong concurrency, easy static binaries for edge/gateway deployment, good crypto libraries.

ADR-003 — UI: REACT

Decision: Build the management/audit console in React.

Rationale: Fast path to a usable first console that consumes the REST API; widely supported.

ADR-004 — AI: LOCAL STATISTICAL ANOMALY DETECTION + OLLAMA AUDIT ASSISTANT (IMPLEMENTED)

Decision: Detect anomalies with statistical methods on attestation metadata inside the core, off the critical path. Provide natural-language audit assistance with a local Ollama model, grounded on facts retrieved from the database (RAG-lite) via Ollama's HTTP API directly. Anomaly detection is never done by the LLM.

Rationale: LLMs are unsuited to high-throughput per-event anomaly scoring but well suited to phrasing answers over retrieved facts. Statistical detection is cheap, explainable, and runs off the critical path. Calling Ollama's HTTP API directly keeps the stack Java-only and avoids a separate Python service.

Note on wording: earlier drafts said "Ollama + LangChain". The implementation uses direct Ollama HTTP with database-side retrieval rather than the Python LangChain framework; LangChain4j or a Python RAG service remain options if richer orchestration or a vector store is later required.

Later enhancements: a semantic vector store over audit logs; moving-baseline / z-score anomaly methods.

ADR-005 — DEPLOYMENT: DOCKER + JENKINS ON DEBIAN

Decision: Package all services as Docker containers; CI/CD with Jenkins; runtime on a single Debian host.

Rationale: Matches available environment and skills; Kubernetes deferred until the core is stable.

Known limitation: The single-host, single-PostgreSQL baseline is not highly available and does not meet the 99.9 % design target; high-availability options are a later phase (consistent with PID 3.3, 5.4, 5.7).

ADR-006 — PROVENANCE & ANCHORING: MERKLE + RFC 3161, NO BLOCKCHAIN

Decision: Establish provenance with in-core hash-linked / Merkle chaining, and make it tamper-evident against the operator by periodically anchoring Merkle roots to one or more independent RFC 3161 timestamping authorities. Do not use a blockchain or distributed ledger.

Rationale: The trust requirement — proving data was not altered between source and decision point, and that the operator did not rewrite history — is met by source signatures plus external anchoring. Anchoring adds constant cost per period, independent of attestation throughput, and needs no consensus protocol or ledger operation. A blockchain would add throughput limits, latency, and operational complexity without solving anything the anchor does not, and a single-operator chain would not decentralize trust anyway.

Alternatives rejected: (a) Permissioned/public blockchain — disproportionate cost and complexity for a solo MVP; genuine decentralization requires a multi-party consortium that does not yet exist. (b) Database permissions alone — insufficient, since a privileged operator can bypass them; the external anchor is what closes that gap.

Later-phase option: Stronger anchoring via multi-witness co-signing or a transparency-log deployment, if a consortium of independent parties emerges.

ADR-007 — SCHEMA VIA FLYWAY; RUNTIME APPEND-ONLY VIA A NON-OWNER ROLE

Decision: Manage the database schema with versioned Flyway migrations and run Hibernate in validate mode (no auto-DDL). Enforce append-only on attestations, merkle_roots, and merkle_leaves by having the application connect as a non-owner role with SELECT/INSERT only; migrations run as the owner role.

Rationale: Auto-DDL (ddl-auto: update) cannot reliably express constraints, the append-only intent, or controlled evolution, and silently drifts from the documented model. Flyway makes the schema explicit, reviewable, and reproducible. A non-owner runtime role is the only way to actually enforce append-only, since table owners bypass table privileges.

Alternatives rejected: (a) Hibernate auto-DDL — convenient but unreviewable and unable to enforce append-only. (b) Single DB role for both migrations and runtime — cannot enforce append-only (owner bypasses grants).

Scope note: Append-only is defense-in-depth; the primary guarantee against operator tampering remains the Merkle chain plus external anchor (ADR-006, M6).

ADR-008 — API SECURITY: SPRING SECURITY WITH JWT + READ KEY (IMPLEMENTED)

Decision: Protect the API with Spring Security. Write/admin operations require a JWT (HS256, role claim; INGEST/ADMIN) issued by /auth/token; read operations accept a static read key. Stateless, no sessions; CORS configurable.

Rationale: The trust chain is only as strong as who may register sources and submit attestations; an open API undermines it. JWT gives role-scoped, stateless write auth; a read key is a low-privilege credential suitable for a UI/proxy. Both are simple to operate for a solo MVP.

Alternatives rejected: (a) No auth — unacceptable for a trust system. (b) API keys only — no standard role/expiry semantics for writes. (c) Full OIDC now — disproportionate for the MVP (a documented later step).

ADR-009 — ANCHOR TOKEN VERIFICATION & TRUST-PINNING (IMPLEMENTED)

Decision: Verify each RFC 3161 anchor token's message imprint, CMS signature, and timestamping ECU; optionally trust-pin the signer to a configured TSA CA via PKIX path building. Verify at acquisition and on read.

Rationale: Checking only imprint/nonce leaves a MITM able to substitute a self-consistent bogus token. Full signature verification closes that; trust-pinning to a pinned CA makes the anchor's provenance verifiable.

Later enhancements: CRL/OCSP revocation checking; multiple pinned CAs for multi-TSA anchoring.

ADR-010 — ABUSE RESISTANCE: IN-APP RATE LIMITING + BODY-SIZE CAP (IMPLEMENTED)

Decision: Enforce a per-client (IP) token-bucket rate limit — general plus a stricter limit for token issuance — and a request-body size cap, in a filter that runs before the security chain. Exceeding returns 429 (with Retry-After) / 413.

Rationale: The single-host MVP needs basic DoS and brute-force protection at the app, without extra infrastructure. Running before security also protects the public token endpoint.

Limitations: In-memory per instance (a shared store such as Redis is required for multi-instance); the body cap is Content-Length based (streaming/chunked bodies need a streaming cap); client-IP correctness depends on the proxy setting X-Forwarded-For.

ADR-011 — ENFORCED APPEND-ONLY VIA A RUNTIME NON-OWNER ROLE, PROVISIONED BY MIGRATION (IMPLEMENTED)

Decision: The application connects as a non-owner role (gridveritas_app) with SELECT/INSERT on append-only tables (and UPDATE on sources only); migrations run as the owner. The role and grants are created by a versioned Flyway migration rather than the Postgres init hook.

Rationale: A table owner bypasses table privileges, so append-only can only be enforced against a non-owner. Provisioning via a versioned migration is deterministic and independent of volume/init timing, unlike docker-entrypoint-initdb.d, which only runs on first initialization and proved unreliable across redeploys.

Note: This hardens the runtime path and a compromised app process; it does not constrain the owner (needed for DDL). The primary guard against deliberate operator tampering remains the external anchor (ADR-006).

5. IMPLEMENTATION SEQUENCE (FROM THIS BASELINE)

Recommended order after this document is accepted as the working baseline. The mapping to PID milestones (M1–M9) is shown in brackets.

- Refine the domain brief with any new findings from utility/ISO conversations or system observation. [M1]
- Spring Boot Core MVP: project skeleton; PostgreSQL schema for sources + attestations + merkle_roots (per 3.3); Merkle provenance (hash-linked roots) implemented; POST/GET attestation and basic verify endpoints; container image; Jenkins pipeline. [M2–M3]
- React UI v1: list attestations, trigger verification, show basic status. [M4]
- Go Edge Agent v1: sign sample data, submit to core, demonstrate end-to-end path. [M5]
- External anchoring & tamper demo: submit Merkle roots to an RFC 3161 TSA; store anchors; expose the proof endpoint (3.4); demonstrate that altering a stored attestation is detected via anchor mismatch. [M6]
- AI layer (done): statistical anomaly detection on attestation metadata (off critical path) plus an Ollama-backed audit assistant (RAG-lite over retrieved facts). [M7]
- Hardening & pilot packaging (done): threat model; API authentication (ADR-008); anchor-token verification (ADR-009); rate limiting + body-size cap (ADR-010); append-only enforced via a runtime non-owner role provisioned by migration (ADR-011); a persisted verification/audit trail; and opt-in mTLS for ingest. Remaining for the pilot: revocation checking, secrets management, and multi-instance concerns. [M8]
- Demo & product polish (in progress): chain-of-custody proof view, a provenance-integrity check that recomputes a stored record's leaf and compares it to the anchored value, a reproducible tamper demo, an overview dashboard, and CI dependency/image scanning. [M10]

6. OPEN POINTS FOR DOMAIN DISCOVERY

These questions should be answered or narrowed during M1 activities:

- Which specific data points (e.g., frequency, voltage, breaker status, meter reads) are highest value for continuous attestation?
- Where can an agent realistically run (gateway, RTU, historian-side, virtual appliance)?
- What existing PKI or identity systems must be integrated?
- What audit artifacts do compliance and OT security teams actually need today?
- At what cadence should Merkle roots be anchored (e.g., per minute, per hour)? This trades anchoring overhead against the maximum window in which undetected tampering could occur, and depends on the domain's audit and latency expectations.
- Which timestamping authority (or authorities) would a regulated partner accept as an external anchor — public RFC 3161 TSAs, an internal/enterprise TSA, or a specific accredited provider?
- What is the realistic path to a first pilot environment (lab, simulation, or limited production path)?

7. IMMEDIATE NEXT STEPS

- Treat this document as the living Domain + Architecture baseline, aligned with the Project Initiation Document (v0.9.1).
- Record the outstanding ADRs — in particular ADR-006 (Provenance & anchoring: Merkle + RFC 3161, no blockchain) — and keep them under the change control defined in Section 4.
- Start or continue domain-discovery conversations / system mapping and update Section 2; prioritize the open points in Section 6 (attestation targets, agent placement, anchoring cadence, acceptable TSA).
- Create the Spring Boot project skeleton and the PostgreSQL schema matching Section 3.3 — including the merkle_roots and anchors tables and the append-only (INSERT/SELECT-only) application role.
- Implement the minimum REST surface in Section 3.4 — including the proof-retrieval endpoint — and containerize it.
- Stand up external anchoring against an RFC 3161 TSA early enough to demonstrate the tamper-detection path (M6).
- Record any further architecture change as a new ADR.

8. REVISION HISTORY

Version 0.1 — July 2026 — Initial Domain Brief & Architecture Baseline created from Project Initiation Document v0.9 and agreed tech stack. Establishes M1/M2 foundation for implementation.

Version 0.2 — July 2026 — Consistency and technical-detail pass, aligned with Project Initiation Document v0.9.1. Key changes:

- Provenance & anchoring made concrete. Added external anchoring of Merkle roots (RFC 3161) as an architectural principle (3.1), a core responsibility and external component (3.2), a security mechanism with three-step verification and the anchor-time vs. measurement-time note (3.5), a new ADR (ADR-006), an implementation step (5), and open domain questions on anchoring cadence and acceptable TSA (6). Explicitly not a blockchain.
- Data model expanded (3.3). Fleshed out attestations (split `src_timestamp` / `received_at`, added `leaf_hash`, `UNIQUE(source_id, sequence)`) and `merkle_roots` (`prev_root_hash`, covered range); added the new anchors table; specified the append-only (INSERT/SELECT-only) application role and noted that the external anchor — not DB permissions — is what protects against operator tampering.
- REST surface (3.4). Added a proof-retrieval endpoint (`GET /attestations/{id}/proof`) and a root/anchor endpoint, so independent parties can verify without trusting the core.
- AI layer corrected. Separated statistical / lightweight-ML anomaly detection (off critical path) from LLM-based natural-language audit assistance (Ollama + LangChain, RAG); ADR-004 rescoped accordingly (3.2, ADR-004).
- ADRs. Added ADR-006 (provenance & anchoring, no blockchain); recorded the rejected Rust alternative and noted this document as the authoritative source for the edge-agent language (ADR-002); added the single-host / single-PostgreSQL availability limitation to ADR-005.
- Domain brief. NERC/FERC roles and NIST versions aligned with the PID (CSF 2.0, SP 800-82 Rev. 3); removed the duplicated problem statement from 2.1; clarified the assumption-vs-dependency relationship with PID 9.3 (2.1, 2.2, 2.5).
- Editorial. Punctuation, cross-references, and cover version corrected (0.8 → 0.2).
- Specified the Merkle leaf/node hashing (RFC 6962 domain separation, 0x00/0x01 prefixes) in 3.5, and replaced the `merkle_roots` id-range with an explicit `merkle_leaves` membership table in 3.3, matching the implemented sealing and proof logic.
- Introduced Flyway-managed schema (validate mode) with baseline migration V1, a two-role append-only model (owner vs. non-owner application role) provisioned via a DBA script, and recorded the decision as ADR-007 (4).

- Implemented the AI layer (M7): statistical anomaly detection into anomaly_findings (3.3, 3.4) and a local Ollama audit assistant grounded on retrieved facts; updated ADR-004 to reflect direct-Ollama RAG-lite instead of Python LangChain.
- Began M8 hardening: added the threat model & cryptographic-design deliverable, API authentication (Spring Security + JWT + read key, governed source registration; ADR-008), and RFC 3161 anchor-token verification with optional CA trust-pinning (ADR-009).
- Completed M8 hardening: verification/audit trail (3.3, 3.4), rate limiting and body-size cap (ADR-010), enforced two-role append-only via versioned migration (ADR-011), and opt-in agent→core mTLS; endpoints and the security outline updated accordingly.

Version 03. — August 2026 — Began M10 (demo & product polish): a chain-of-custody proof view, an anchored-provenance integrity check (provenanceIntact), a reproducible tamper demonstration, and CI scanning. No change to the trust model.